



LeetCode HOT 100

PART 8

Binary Tree

二叉树



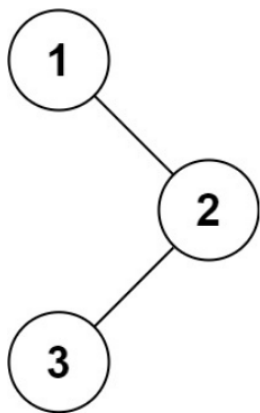


36. 二叉树的中序遍历 (No.94)

EASY

给定一个二叉树的根节点 `root`，返回 它的 **中序** 遍历。

示例 1:



输入: `root = [1,null,2,3]`

输出: `[1,3,2]`

示例 2:

输入: `root = []`

输出: `[]`

示例 3:

输入: `root = [1]`

输出: `[1]`

提示:

- 树中节点数目在范围 `[0, 100]` 内
- `-100 <= Node.val <= 100`

进阶: 递归算法很简单，你可以通过迭代算法完成吗？

▶▶ 36. 二叉树的中序遍历 (No.94) EASY

Solution 1 - 递归

思路

按“左根右”顺序遍历。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def backtrack(root):
            if not root:
                return
            backtrack(root.left)
            res.append(root.val)
            backtrack(root.right)

        res = []
        backtrack(root)
        return res
```

时间复杂度: $O(n)$, n 为二叉树的节点数; 空间复杂度: $O(h)$, h 为二叉树的深度

▶▶ 36. 二叉树的中序遍历 (No.94) EASY

Solution 2 - 迭代

思路

用栈显示模拟递归调用的过程：遍历左孩子循环入栈，然后栈顶元素出栈加入res，接着按照从深到浅的顺序遍历这些栈中节点的右孩子重复上述流程。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        stack, res = [], []
        while root or stack:
            while root: # 遍历左孩子
                stack.append(root)
                root = root.left
            root = stack.pop()
            res.append(root.val) # 根节点加入res
            root = root.right # 遍历右孩子
        return res
```

时间复杂度： $O(n)$ ；空间复杂度： $O(h)$



36. 二叉树的中序遍历 (No.94)

EASY

Solution 3 - Morris遍历

思路

Morris遍历的思想主要是利用二叉树中的空闲指针，提前把回溯的路线给接好，从而节省栈存储节点的空间复杂度。

对于每个有左孩子的根节点`root`，提前找好它的前驱`predecessor`，设置`pre.right=root`（反正本来`pre.right=None`右指针空闲）。具体的找法就是从`root.left`开始不断往右找，如果发现尽头是空，说明还在“左孩子入栈”阶段，把最后遍历到的`pre`指向`root`；如果发现尽头是`root`，说明已经在“左孩子出栈”阶段回溯到`root`了（形成了环），那就往右继续遍历。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        res = []
        while root:
            # 有左子树找predecessor (根据pre的定义, pre.right必然为None)
            if root.left:
                pre = root.left
                while pre.right and pre.right != root:
                    pre = pre.right
                # 因为pre.right=None退出循环, 说明还没标记过, 让predecessor右孩子指向root, 继续遍历左孩子
                if not pre.right:
                    pre.right = root
                    root = root.left
                # 因为pre.right=root退出循环, 说明是回溯回到root, 左子树已经遍历完了
            else:
                res.append(root.val)
                pre.right = None # 复原 (非必须)
                root = root.right
            else: # 没有左子树root加入res然后遍历右子树
                res.append(root.val)
                root = root.right
        return res
```

时间复杂度: $O(n)$; 空间复杂度: $O(1)$



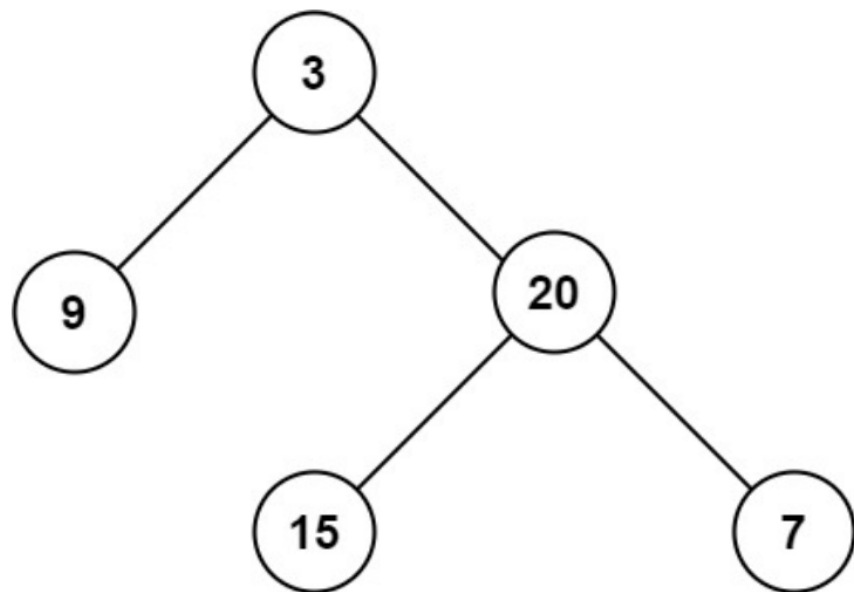
37. 二叉树的最大深度 (No.104)

EASY

给定一个二叉树 `root`，返回其最大深度。

二叉树的 **最大深度** 是指从根节点到最远叶子节点的最长路径上的节点数。

示例 1:



输入: `root = [3,9,20,null,null,15,7]`

输出: 3

示例 2:

输入: `root = [1,null,2]`

输出: 2

提示:

- 树中节点的数量在 `[0, 104]` 区间内。
- `-100 <= Node.val <= 100`

▶▶ 37. 二叉树的最大深度 (No.104)

思路

左孩子和右孩子深度较大值+1就是根节点的深度，边界条件就是空节点深度为0。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        return max(self.maxDepth(root.left), self.maxDepth(root.right)) + 1
```

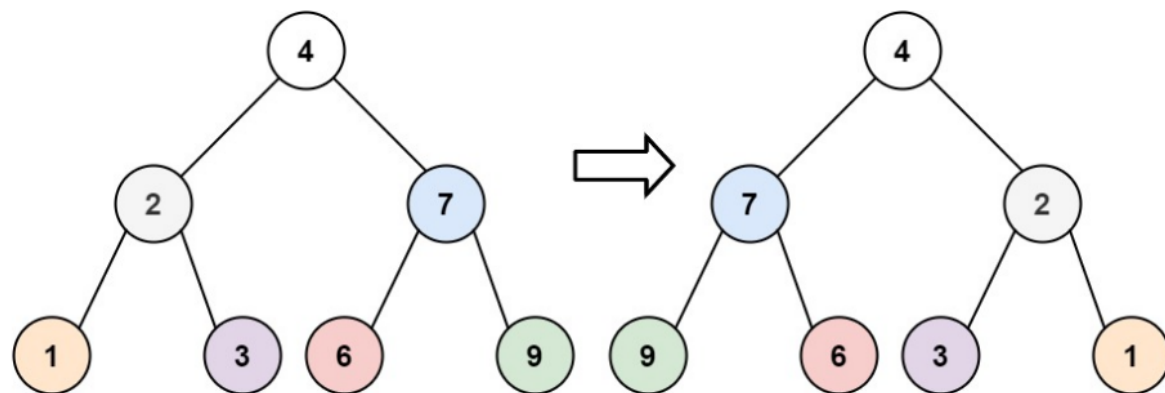
时间复杂度: $O(n)$; 空间复杂度: $O(h)$

38. 翻转二叉树 (No.226)

EASY

给你一棵二叉树的根节点 `root`，翻转这棵二叉树，并返回其根节点。

示例 1:



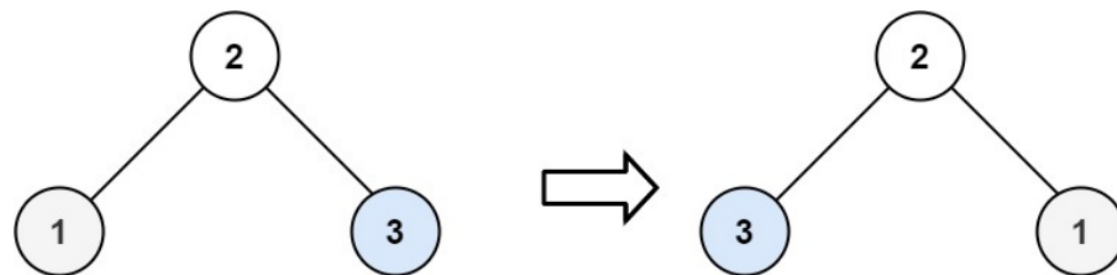
输入: `root = [4,2,7,1,3,6,9]`

输出: `[4,7,2,9,6,3,1]`

提示:

- 树中节点数目范围在 `[0, 100]` 内
- `-100 <= Node.val <= 100`

示例 2:



输入: `root = [2,1,3]`

输出: `[2,3,1]`

示例 3:

输入: `root = []`

输出: `[]`

▶ 38. 翻转二叉树 (No.226)

思路

递归地交换左右孩子。边界条件：空节点直接返回root/None。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if not root:
            return root
        root.left, root.right = self.invertTree(root.right), self.invertTree(root.left)
        return root
```

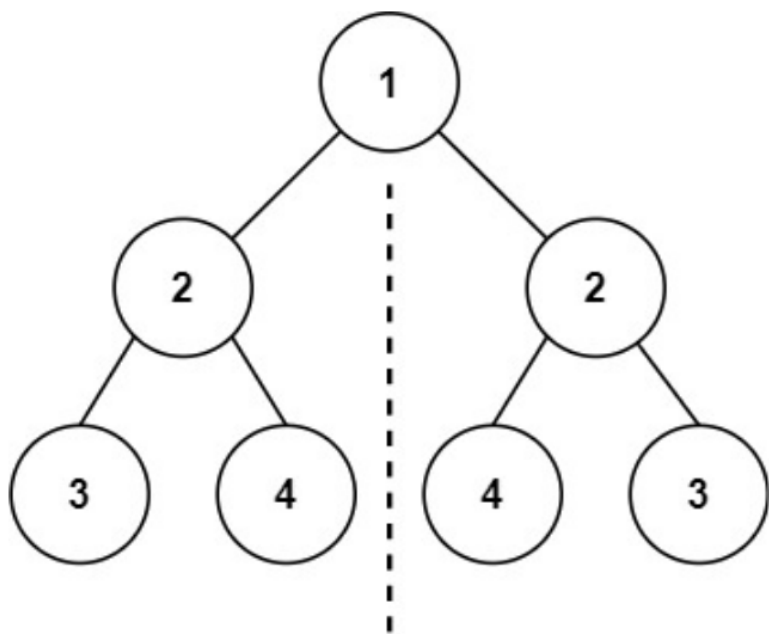
时间复杂度： $O(n)$ ；空间复杂度： $O(h)$

39. 对称二叉树 (No.226)

EASY

给你一个二叉树的根节点 `root`，检查它是否轴对称。

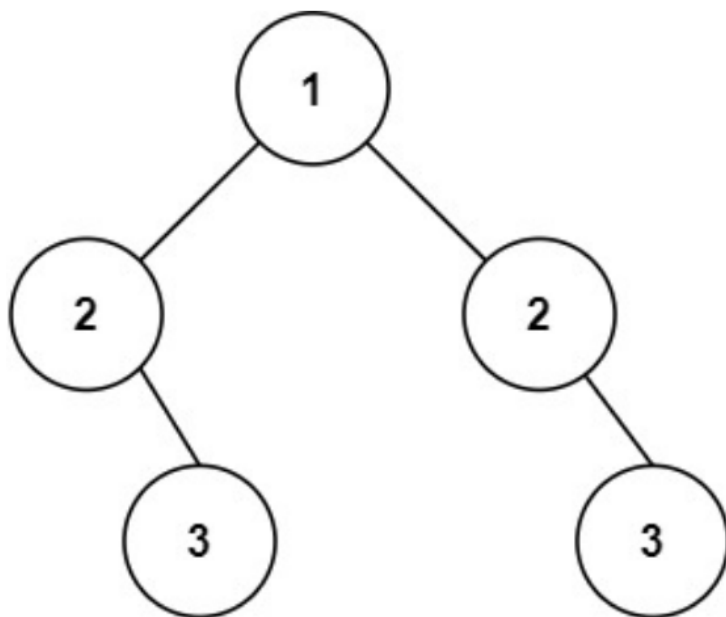
示例 1:



输入: `root = [1,2,2,3,4,4,3]`

输出: `true`

示例 2:



输入: `root = [1,2,2,null,3,null,3]`

输出: `false`

提示:

- 树中节点数目在范围 `[1, 1000]` 内
- `-100 <= Node.val <= 100`

进阶: 你可以运用递归和迭代两种方法解决这个问题吗?

▶ 39. 对称二叉树 (No.226) EASY

Solution 1 - 递归

思路

compare函数用于递归地判断根节点两侧的两棵子树是否对称。

root1和root2对称需满足：二者val相等&&root1左和root2右对称&&root1右和root2左对称。

边界条件：双空节点返回True，单空节点返回False。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def compare(root1, root2):
            # 比较root1和root2子树是否对称 (root1和root2在根节点两侧)
            if not root1 and not root2:
                return True
            if not root1 or not root2: # 第一个None一个不是None
                return False
            return root1.val == root2.val and compare(root1.left, root2.right) and compare(root1.right, root2.left)

        return compare(root.left, root.right) # 题目保证至少1个节点
```

时间复杂度： $O(n)$ ；空间复杂度： $O(h)$

▶ 39. 对称二叉树 (No.226)

EASY

Solution 2 - 迭代

思路

维护一个队列，每次取两个节点判断两棵子树是否对称，判断逻辑和Solution1一致。一旦有问题直接返回False，否则继续把“应当对称”的两个子树放在一块入队。

这里用`deque.popleft()`而不是普通列表的`pop(0)`，因为`pop(0)`通常还需要挪动数组，会造成额外的时间开销。

时间复杂度： $O(n)$ ；空间复杂度： $O(n)$

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        queue = deque([root, root])
        while queue:
            root1 = queue.popleft()
            root2 = queue.popleft()
            # root1和root2都是空节点
            if not root1 and not root2:
                continue
            # root1和root2单空或者val不相等
            if not root1 or not root2 or root1.val != root2.val:
                return False
            # 对称部分要挨在一起入堆
            queue.append(root1.left)
            queue.append(root2.right)
            queue.append(root1.right)
            queue.append(root2.left)
        return True
```

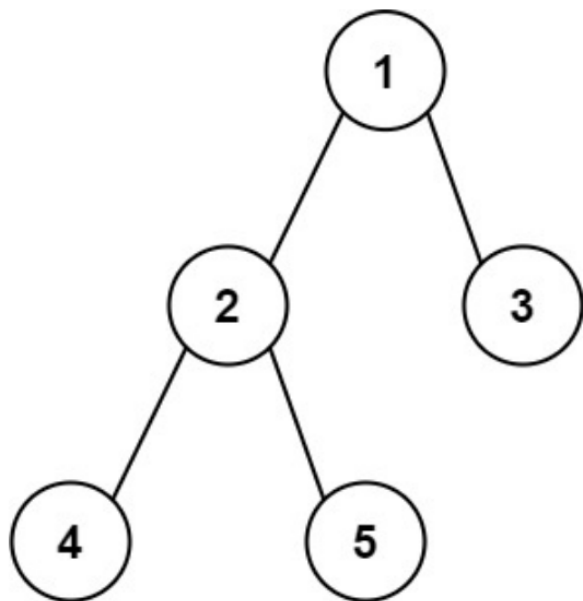
40. 二叉树的直径 (No.543) EASY

给你一棵二叉树的根节点，返回该树的 **直径**。

二叉树的 **直径** 是指树中任意两个节点之间最长路径的 **长度**。这条路径可能经过也可能不经过根节点 `root`。

两节点之间路径的 **长度** 由它们之间边数表示。

示例 1:



输入: `root = [1,2,3,4,5]`

输出: 3

解释: 3，取路径 `[4,2,1,3]` 或 `[5,2,1,3]` 的长度。

示例 2:

输入: `root = [1,2]`

输出: 1

提示:

- 树中节点数目在范围 `[1, 104]` 内
- `-100 <= Node.val <= 100`

▶▶ 40. 二叉树的直径 (No.543)

思路

二叉树的直径必定以某个节点`node`为根，穿过这个根形成直径，直径的长度等于`node.left`和`node.right`的深度之和。因此按照[37. 二叉树的最大深度]写`maxDepth`函数用于返回深度，递归的同时不断计算左右孩子的深度之和来更新直径。

注意`res`在嵌套函数内被赋值，必须声明为全局变量。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        def maxDepth(root):
            # 返回从叶子节点到root的最长路径，过程中更新直径
            if not root:
                return 0
            left_len, right_len = maxDepth(root.left), maxDepth(root.right)
            self.res = max(self.res, left_len + right_len)
            return max(left_len, right_len) + 1

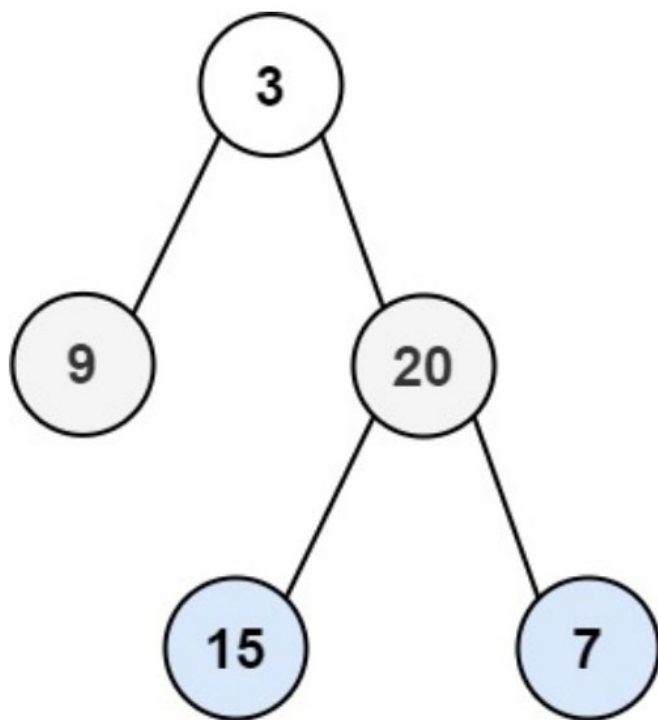
        self.res = 0
        maxDepth(root)
        return self.res
```

时间复杂度: $O(n)$; 空间复杂度: $O(h)$

41. 二叉树的层序遍历 (No.102) MEDIUM

给你二叉树的根节点 `root`，返回其节点值的 **层序遍历**。（即逐层地，从左到右访问所有节点）。

示例 1:



输入: `root = [3,9,20,null,null,15,7]`
输出: `[[3],[9,20],[15,7]]`

示例 2:

输入: `root = [1]`
输出: `[[1]]`

示例 3:

输入: `root = []`
输出: `[]`

提示:

- 树中节点数目在范围 `[0, 2000]` 内
- `-1000 <= Node.val <= 1000`

41. 二叉树的层序遍历 (No.102) MEDIUM

思路

经典的队列实现广度优先搜索（层序遍历），需要每层统计一下节点个数 n ，然后这 n 个节点为一组一起处理，对应的值放入相同列表。

时间复杂度: $O(n)$

空间复杂度: $O(n)$

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []
        queue, res = deque([root]), []
        while queue:
            n = len(queue)  # 这一层的节点个数
            tmp = []
            for _ in range(n):
                node = queue.popleft()
                tmp.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            res.append(tmp)
        return res
```

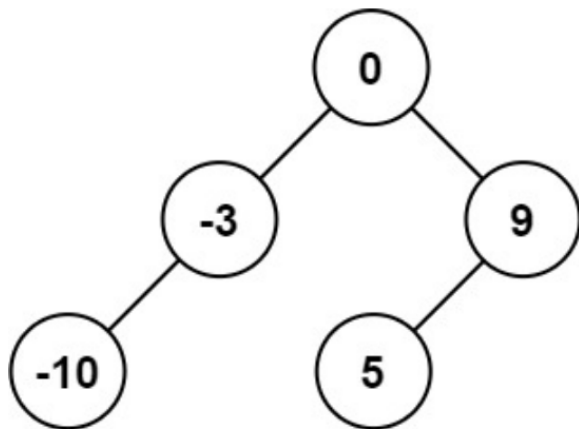



42. 将有序数组转换为二叉搜索树 (No.108)

EASY

给你一个整数数组 `nums`，其中元素已经按 **升序** 排列，请你将其转换为一棵 **平衡** 二叉搜索树。

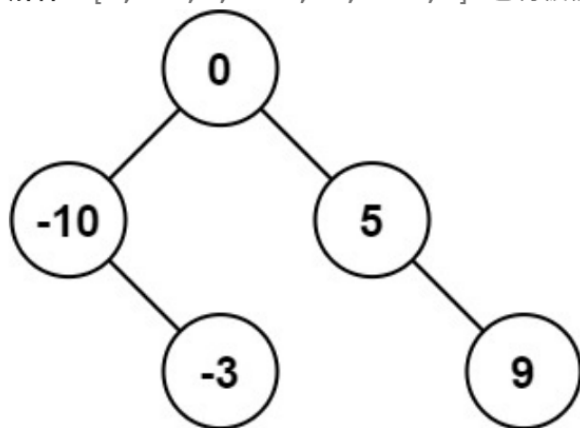
示例 1:



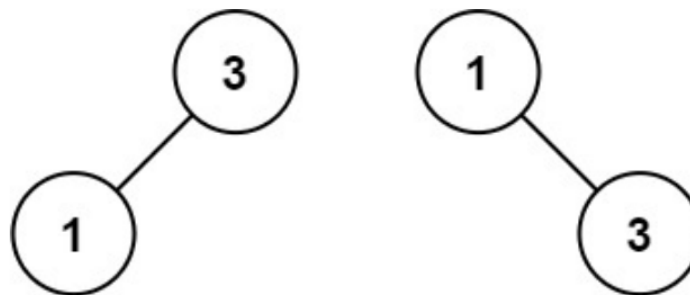
输入: `nums = [-10,-3,0,5,9]`

输出: `[0,-3,9,-10,null,5]`

解释: `[0,-10,5,null,-3,null,9]` 也将被视为正确答案:



示例 2:



输入: `nums = [1,3]`

输出: `[3,1]`

解释: `[1,null,3]` 和 `[3,1]` 都是高度平衡二叉搜索树。

提示:

- `1 <= nums.length <= 104`
- `-104 <= nums[i] <= 104`
- `nums` 按 **严格递增** 顺序排列

► 42. 将有序数组转换为二叉搜索树 (No.108)

思路

将数组一分为二，中点作为根节点，前半部分继续递归构造左子树，后半部分继续递归构造右子树。由于每次都切中点，前后子树节点总数不会超过1，递归构造后保证二叉树的平衡。这里用left和right传递切分的下标，如果使用切片会增加额外的时间复杂度。

时间复杂度: $O(n)$

空间复杂度: $O(n)$

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def buildTree(left, right):
            # nums[left:right+1]构造子树
            if left > right:
                return None
            mid = (left + right) // 2
            root = TreeNode(nums[mid])
            root.left = buildTree(left, mid-1)
            root.right = buildTree(mid+1, right)
            return root

        root = buildTree(0, len(nums)-1)
        return root
```

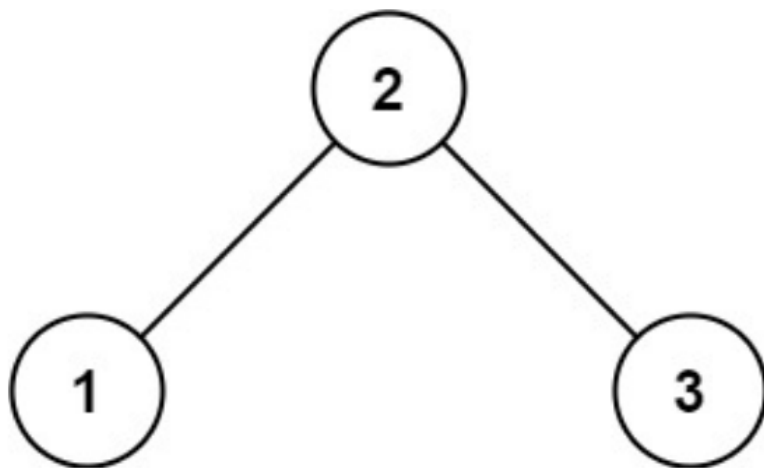
43. 验证二叉搜索树 (No.98) MEDIUM

给你一个二叉树的根节点 `root`，判断其是否是一个有效的二叉搜索树。

有效 二叉搜索树定义如下：

- 节点的左子树只包含 **严格小于** 当前节点的数。
- 节点的右子树只包含 **严格大于** 当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

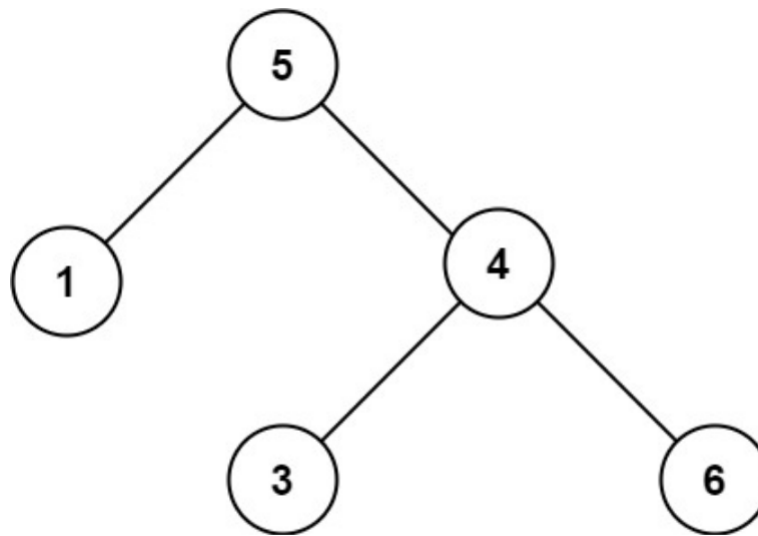
示例 1:



输入: `root = [2,1,3]`

输出: `true`

示例 2:



输入: `root = [5,1,4,null,null,3,6]`

输出: `false`

解释: 根节点的值是 5，但是右子节点的值是 4。

提示:

- 树中节点数目范围在 $[1, 10^4]$ 内
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

▶ 43. 验证二叉搜索树 (No.98) MEDIUM

Solution 1 - 递归

思路

递归判断：左子树节点的值应介于开区间($\text{lower}, \text{root.val}$)之间，右子树节点的值应介于开区间($\text{root.val}, \text{upper}$)之间。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def check(root, lower, upper):
            if not root:
                return True
            if root.val <= lower or root.val >= upper:
                return False
            return check(root.left, lower, root.val) and check(root.right, root.val, upper)
        return check(root, -inf, inf)
```

时间复杂度: $O(n)$; 空间复杂度: $O(h)$

43. 验证二叉搜索树 (No.98) MEDIUM

Solution 2 - 中序遍历

思路

二叉搜索树的中序遍历序列是递增序列，因此进行中序遍历检查序列是否严格递增。

注意`pre, cur, res`在嵌套函数内被赋值，必须声明为全局变量。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def inOrder(root):
            if not root:
                return
            inOrder(root.left)
            self.pre, self.cur = self.cur, root.val
            if self.cur <= self.pre: # 序列非严格递增返回False
                self.res = False
            inOrder(root.right)

        self.pre, self.cur = -inf, -inf
        self.res = True
        inOrder(root)
        return self.res
```

时间复杂度: $O(n)$; 空间复杂度: $O(h)$



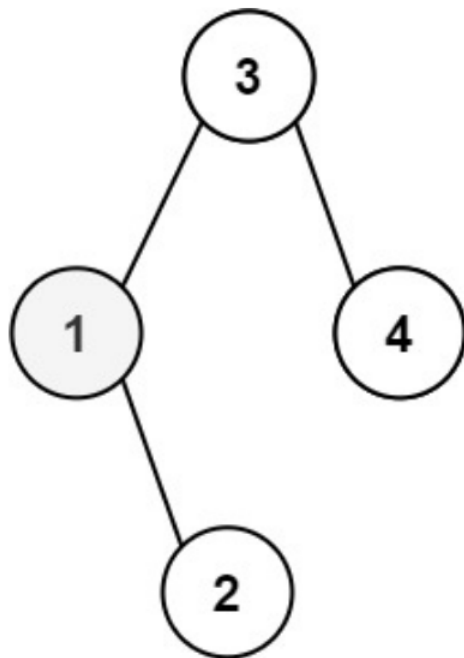
44. 二叉搜索树中第K小的元素 (No.230)



给定一个二叉搜索树的根节点 `root`，和一个整数 `k`，请你设计一个算法查找其中第 `k` 小的元素（`k` 从 1 开始计数）。

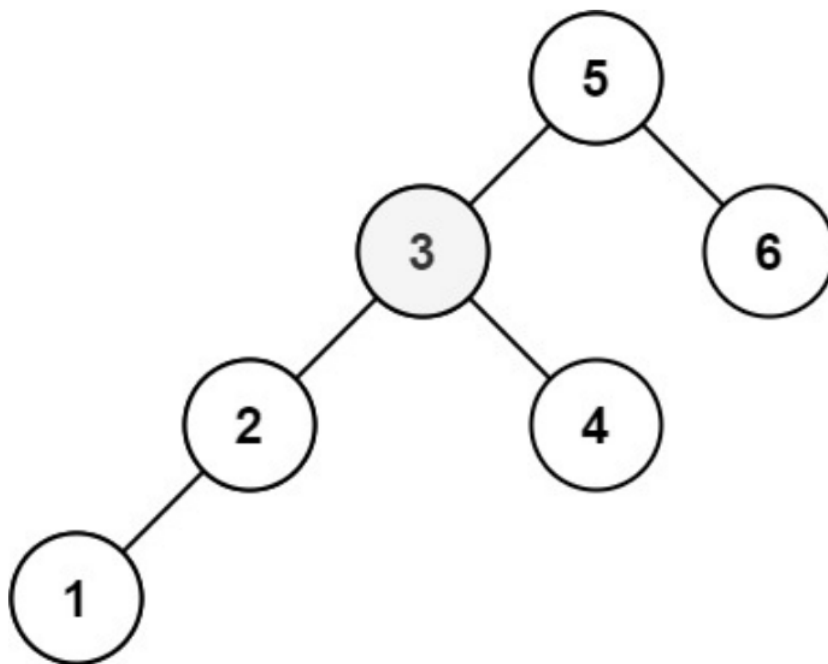
进阶：如果二叉搜索树经常被修改（插入/删除操作）并且你需要频繁地查找第 `k` 小的值，你将如何优化算法？

示例 1:



输入: `root = [3,1,4,null,2]`, `k = 1`
输出: 1

示例 2:



输入: `root = [5,3,6,2,4,null,null,1]`, `k = 3`
输出: 3

提示:

- 树中的节点数为 `n`。
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

44. 二叉搜索树中第K小的元素 (No.230)



Solution 1 - 中序遍历

思路

利用二叉搜索树的中序遍历序列是递增序列，返回中序遍历的第k个元素。官解给了迭代的写法，可以提前返回结果。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        def inOrder(root):
            if not root:
                return
            inOrder(root.left)
            self.cnt += 1
            if self.cnt == k:
                self.res = root.val
            inOrder(root.right)

        self.cnt = 0
        self.res = root.val
        inOrder(root)
        return self.res
```

时间复杂度: $O(n)$; 空间复杂度: $O(h)$

44. 二叉搜索树中第K小的元素 (No.230)



Solution 2 - 记录子树节点数

思路

如果需要频繁修改二叉搜索树并访问第k小元素，更好的做法是维护一个哈希表，key=节点，value=子树节点总数num。这样增删节点的时候，只需要沿路径修改num就可以了。

先用一趟遍历记录node_map保存所有节点的子树节点数，再用一个循环找到第k小的节点。如果当前节点左孩子的子树节点数不到k-1，第k小节点就在右子树；大于k-1，第k小节点在左子树；等于k-1，第k小节点就是root。

官解还给了一个AVL树实现，实际面试手搓AVL树也太逆天了，那就是存心要挂人了。

时间复杂度： $O(n)$ ；空间复杂度： $O(n)$ ，因为用了哈希表

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        def countNode(root): # 求root子树的节点数
            if not root:
                return 0
            node_map[root] = countNode(root.left) + countNode(root.right) + 1
            return node_map[root]

        node_map = {}
        countNode(root) # 把node_map填完
        # 查找第k小的数（题目保证至少1个节点）
        while root:
            left_num = node_map[root.left] if root.left else 0
            if left_num < k - 1: # 第k小节点在右子树
                root = root.right
                k -= left_num + 1
            elif left_num > k - 1: # 第k小节点在左子树
                root = root.left
            else: # 第k小节点就是root
                return root.val
```


45. 二叉树的右视图 (No.230) MEDIUM

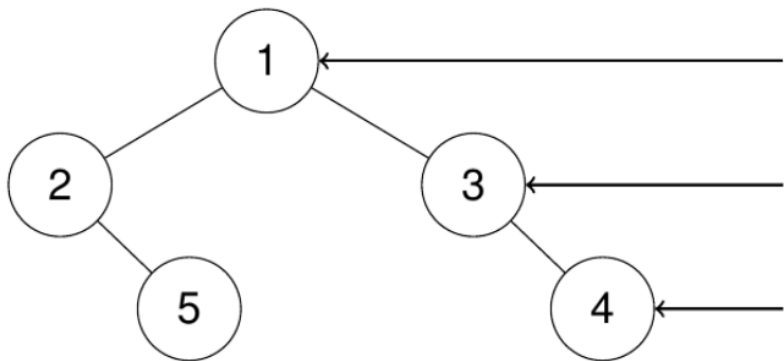
给定一个二叉树的 **根节点** `root`，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

示例 1:

输入: `root = [1,2,3,null,5,null,4]`

输出: `[1,3,4]`

解释:

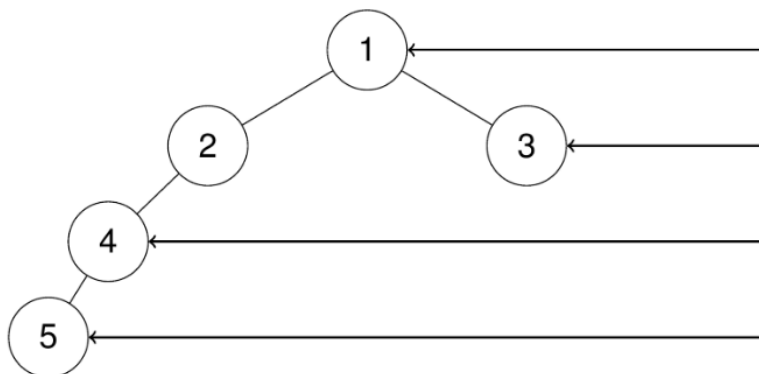


示例 2:

输入: `root = [1,2,3,4,null,null,null,5]`

输出: `[1,3,4,5]`

解释:



示例 3:

输入: `root = [1,null,3]`

输出: `[1,3]`

示例 4:

输入: `root = []`

输出: `[]`

提示:

- 二叉树的节点个数的范围是 `[0,100]`
- `-100 <= Node.val <= 100`

45. 二叉树的右视图 (No.230)

Solution 1.1 - 深度优先(递归)

思路

用“先遍历右子树的中序遍历”遍历整棵树，这样每到新的一个深度第一个遍历的节点就是最右边的节点。用一个哈希表key=深度，value=最右节点来记录每层的最右节点，最后把它们按顺序返回。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        def inOrder(root, depth): # 右子树优先的中序遍历
            if not root:
                return
            inOrder(root.right, depth+1)
            if depth not in node_map: # 这一层的最右节点还没找到
                node_map[depth] = root
            inOrder(root.left, depth+1)

        node_map = {}
        inOrder(root, 0)
        return [node_map[x].val for x in sorted(node_map)]
```

时间复杂度: $O(n)$; 空间复杂度: $O(n)$, 因为用了哈希表

45. 二叉树的右视图 (No.230) MEDIUM

Solution 1.2 - 深度优先(迭代)

思路

同样深度优先的写法，再用迭代写一次右子树优先的中序遍历，可以加深中序遍历两种写法的理解，压栈的时候depth作为节点信息也要压栈。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        stack, node_map = [], {}
        depth = 0
        while root or stack: # 右子树优先的中序遍历
            while root:
                stack.append((root, depth)) # 深度也要压栈
                root = root.right
                depth += 1
            root, depth = stack.pop()
            if depth not in node_map:
                node_map[depth] = root.val
            root = root.left
            depth += 1
        return [node_map[x] for x in sorted(node_map)]
```

时间复杂度: $O(n)$; 空间复杂度: $O(n)$, 因为用了哈希表



45. 二叉树的右视图 (No.230)



Solution 2 - 广度优先

思路

层序遍历，和[41. 二叉树的层序遍历]逻辑一致，然后只需要记录每层最后一个节点。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        if not root:
            return []
        queue, res = deque([root]), []
        while queue:
            n = len(queue)
            for _ in range(n):
                node = queue.popleft()
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            res.append(node.val)
        return res
```

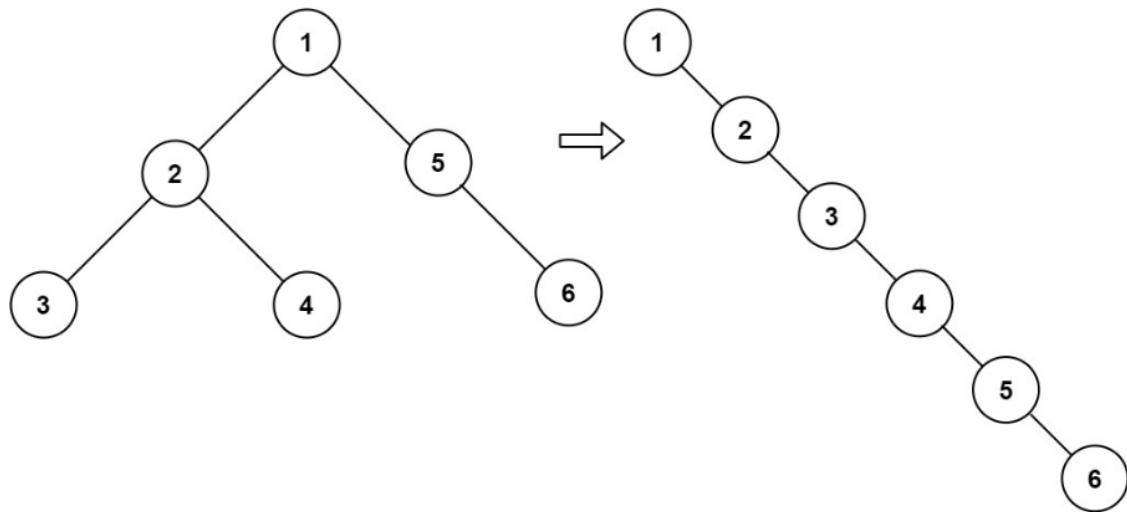
时间复杂度: $O(n)$; 空间复杂度: $O(n)$

46. 二叉树展开为链表 (No.114) MEDIUM

给你二叉树的根结点 `root`，请你将它展开为一个单链表：

- 展开后的单链表应该同样使用 `TreeNode`，其中 `right` 子指针指向链表中下一个结点，而左子指针始终为 `null`。
- 展开后的单链表应该与二叉树 **先序遍历** 顺序相同。

示例 1：



输入: `root = [1,2,5,3,4,null,6]`

输出: `[1,null,2,null,3,null,4,null,5,null,6]`

示例 2：

输入: `root = []`

输出: `[]`

示例 3：

输入: `root = [0]`

输出: `[0]`

提示：

- 树中结点数在范围 `[0, 2000]` 内
- `-100 <= Node.val <= 100`

进阶：你可以使用原地算法（`O(1)` 额外空间）展开这棵树吗？

46. 二叉树展开为链表 (No.114) MEDIUM

Solution 1 - 先序遍历

思路

用栈实现先序遍历的迭代写法，并且能和拼接链表的操作合在一起写，边遍历边创建链表。

具体做法是，每个遍历的`root`接上链表，把右子树压栈后，`root`继续向左子树更深处遍历。如果`root`遍历到头了，再弹栈回溯到上一个右子树。

时间复杂度： $O(n)$ ；空间复杂度： $O(h)$

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.
        """
        stack, tail = [], TreeNode() # tail是尾节点，初始为虚拟节点
        while root or stack:
            if not root:
                root = stack.pop()
            tail.right = root
            if root.right: # 右子树压栈
                stack.append(root.right)
            left_node = root.left
            root.left = None # 左孩子置空
            tail, root = root, left_node # 继续向左子树遍历
```

46. 二叉树展开为链表 (No.114) MEDIUM

Solution 2 - Morris遍历

思路

和[36. 二叉树的中序遍历]一样，用Morris遍历可以把空间复杂度降到 $O(1)$ 。这题先序Morris遍历比之前中序Morris遍历简单，因为pre直接连到右子树就可以了，不需要担心成环。

具体来说，如果root有左子树，就从其左孩子开始向右遍历，最后1个节点就是pre，并且下一个遍历的节点就是root的左孩子；如果没有左子树，直接开始遍历root的右子树。

时间复杂度： $O(n)$ ；空间复杂度： $O(1)$

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.
        """
        while root:
            left_node, right_node = root.left, root.right
            root.left = None
            if left_node: # 有左子树需要找predecessor
                pre = left_node
                while pre.right:
                    pre = pre.right
                pre.right = right_node
                root.right = left_node # 链表下一个节点连上
                root = left_node
            else: # 没有左子树直接到右子树
                root.right = right_node # 链表下一个节点连上
                root = right_node
```

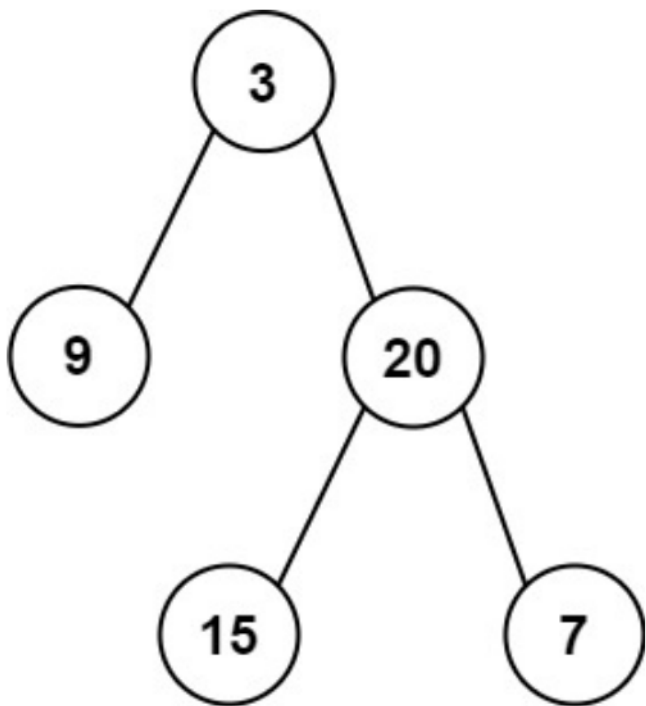


47. 从前序和中序遍历序列构造二叉树 (No.105)



给定两个整数数组 `preorder` 和 `inorder`，其中 `preorder` 是二叉树的先序遍历，`inorder` 是同一棵树的中序遍历，请构造二叉树并返回其根节点。

示例 1:



输入: `preorder = [3,9,20,15,7]`, `inorder = [9,3,15,20,7]`
输出: `[3,9,20,null,null,15,7]`

示例 2:

输入: `preorder = [-1]`, `inorder = [-1]`
输出: `[-1]`

提示:

- `1 <= preorder.length <= 3000`
- `inorder.length == preorder.length`
- `-3000 <= preorder[i], inorder[i] <= 3000`
- `preorder` 和 `inorder` 均无重复元素
- `inorder` 均出现在 `preorder`
- `preorder` 保证为二叉树的前序遍历序列
- `inorder` 保证为二叉树的中序遍历序列

47. 从前序和中序遍历序列构造二叉树 (No.105) MEDIUM Solution 1.1 - 递归(切片)

思路

每次preorder第一个数就是根节点，找到它在inorder中的下标root_index就是左子树的个数，因此inorder在这之前全是左子树中序序列，preorder第1个元素后面root_index个数是左子树的前序序列；除此之外是右子树的部分，如此递归下去。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        if not preorder:
            return None
        if len(preorder) == 1: # 只有一个节点
            return TreeNode(preorder[0])
        root = TreeNode(preorder[0])
        # 左子树的节点个数就是根节点在inorder中的下标
        root_index = inorder.index(root.val)
        # 左子树: preorder[1:root_index+1]+inorder[:root_index]
        root.left = self.buildTree(preorder[1:root_index+1], inorder[:root_index])
        # 右子树: preorder[root_index+1:]+inorder[root_index+1:]
        root.right = self.buildTree(preorder[root_index+1:], inorder[root_index+1:])
        return root
```

时间复杂度： $O(n \log n)$ ，切片时间 $O(n)$ ，且当二叉树是一条链时最坏达到 $O(n^2)$ ；空间复杂度： $O(n)$ ，因为有切片产生的临时列表

47. 从前序和中序遍历序列构造二叉树 (No.105) MEDIUM Solution 1.2 - 递归(双指针)

思路

为了避免切片带来的额外开销，使用双指针在数组中移动，降低时间复杂度。

在算根节点下标的时候要计算的是偏移量

offset（注意要减去in_left），不过整体看起来写得更复杂了一点。

本题官解还给了一种迭代写法，可读性很差，就不写了。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def buildWithIndex(pre_left, pre_right, in_left, in_right):
            # 用preorder[pre_left:pre_right+1]和inorder[in_left:in_right+1]构造二叉树
            if pre_left > pre_right:
                return None
            if pre_left == pre_right:
                return TreeNode(preorder[pre_left])
            root = TreeNode(preorder[pre_left])
            # 左子树的节点个数就是根节点在inOrder中的下标
            offset = num_map[root.val] - in_left # root在inorder中的下标偏移
            # 左子树: preorder[pre_left+1:pre_left+root_index+1]+inorder[in_left:in_left+root_index]
            root.left = buildWithIndex(pre_left+1, pre_left+offset, in_left, in_left+offset-1)
            # 右子树: preorder[pre_left+root_index+1:pre_right+1]+inorder[in_left+root_index+1:in_right+1]
            root.right = buildWithIndex(pre_left+offset+1, pre_right, in_left+offset+1, in_right)
            return root

        num_map = {} # key是数值, value是它在inorder里的下标
        pre_len, in_len = len(preorder), len(inorder)
        for i in range(in_len):
            num_map[inorder[i]] = i
        root = buildWithIndex(0, pre_len-1, 0, in_len-1)
        return root
```

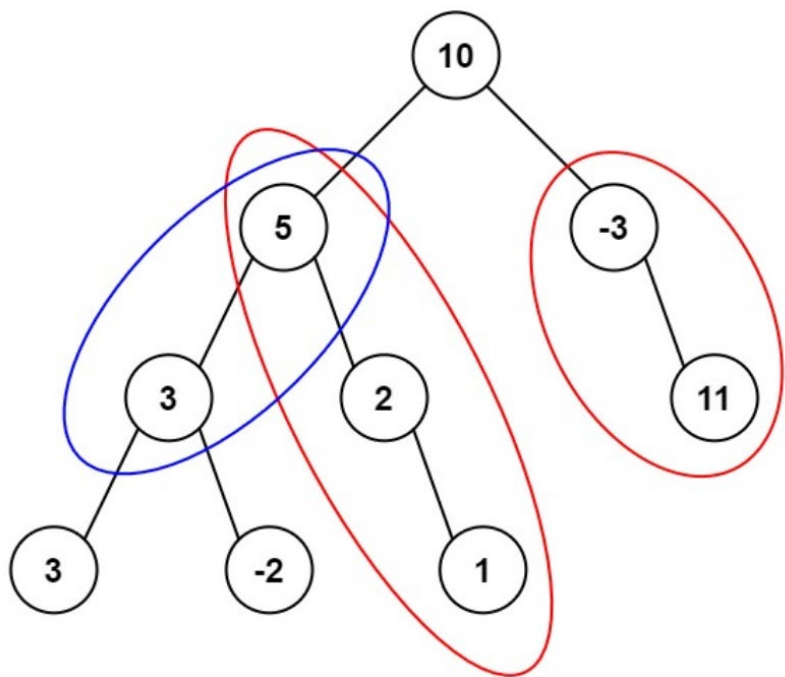
时间复杂度: $O(n)$, 因为填num_map遍历了inorder; 空间复杂度: $O(n)$, 因为用了哈希表

48. 路径总和 (No.437) MEDIUM

给定一个二叉树的根节点 `root`，和一个整数 `targetSum`，求该二叉树里节点值之和等于 `targetSum` 的路径的数目。

路径 不需要从根节点开始，也不需要叶子节点结束，但是路径方向必须是向下的（只能从父节点到子节点）。

示例 1:



示例 2:

输入: `root = [5,4,8,11,null,13,4,7,2,null,null,5,1]`, `targetSum = 22`
输出: 3

提示:

- 二叉树的节点个数的范围是 `[0,1000]`
- `-109 <= Node.val <= 109`
- `-1000 <= targetSum <= 1000`

输入: `root = [10,5,-3,3,2,null,11,3,-2,null,1]`, `targetSum = 8`

输出: 3

解释: 和等于 8 的路径有 3 条，如图所示。

48. 路径总和 (No.437) MEDIUM

思路

这题中的路径其实就是根节点到某叶子节点数组的子数组之和，因此可以用前缀和实现。这题跟[10.和为K的子数组]几乎一模一样，仍然是用一个哈希表key=前缀和+target，当发现(前缀和1)在哈希表里，说明前面有个(前缀和0)满足：

(前缀和0)+target)=(前缀和1)，

因此必有子数组和为targetSum，就找到了一条路径。

只不过之前只有1个分支，这道题由于是二叉树有两个分支，因此先序遍历回溯时需要撤销对哈希表的更新进行复原。

时间复杂度： $O(n)$

空间复杂度： $O(n)$ ，如果回溯时对于计数为0的key用del手动删，可以保证哈希表key数量不超过h，从而时间复杂度是 $O(h)$

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        def preOrder(root, summ):
            if not root:
                return
            summ += root.val
            # 前面某个前缀和+target=当前前缀和，说明中间一段路程是target
            if summ in num_map:
                self.res += num_map[summ]
            num_map[summ + targetSum] += 1
            preOrder(root.left, summ)
            preOrder(root.right, summ)
            num_map[summ + targetSum] -= 1 # 回溯时该分支要去掉

        # key=和target的差距，value=计数这样的路径有多少条
        num_map = Counter()
        num_map[targetSum] += 1
        self.res = 0
        preOrder(root, 0)
        return self.res
```



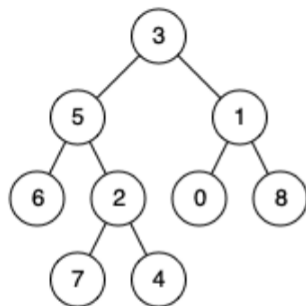
49. 二叉树的最近公共祖先 (No.236)

MEDIUM

给定一个二叉树, 找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为：“对于有根树 T 的两个节点 p、q，最近公共祖先表示为一个节点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

示例 2:

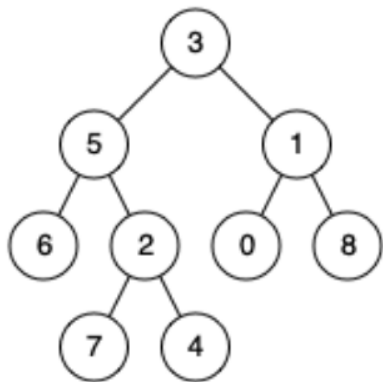


输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

输出: 5

解释: 节点 5 和节点 4 的最近公共祖先是节点 5。因为根据定义最近公共祖先节点可以为节点本身。

示例 1:



输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

输出: 3

解释: 节点 5 和节点 1 的最近公共祖先是节点 3。

示例 3:

输入: root = [1,2], p = 1, q = 2

输出: 1

提示:

- 树中节点数目在范围 $[2, 10^5]$ 内。
- $-10^9 \leq \text{Node.val} \leq 10^9$
- 所有 Node.val 互不相同
- $p \neq q$
- p 和 q 均存在于给定的二叉树中。

49. 二叉树的最近公共祖先 (No.236) MEDIUM

思路

只需要回传4种节点：None，p，q，pq交汇点。当交汇点还没出现时遇到p或q就回传，如果当前节点不是p或q，left_find和right_find也至多只有1个非None，因此返回这个分支的结果。假如之后回传结束也没找到交汇点，说明p和q是祖先关系，此时回传的是p和q辈分更大的一个，恰好就是二者最近祖先；如果找到交汇点会回传交汇点，此时说明p和q并非祖先关系，二者最近祖先就是交汇点（也是整棵树唯一满足“p和q分别在其左子树和右子树上”的节点）。

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        # root为空或p或q直接返回
        if not root or root == p or root == q:
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        # 同时找到了返回交汇点，否则返回不为None的那个
        # 如果都是None定义返回right，没影响，因为说明p和q不在这棵子树上
        if left and right:
            return root
        return left if left else right
```

时间复杂度： $O(n)$ ；空间复杂度： $O(h)$

50. 二叉树中的最大路径和 (No.124) HARD

二叉树中的 **路径** 被定义为一条节点序列，序列中每对相邻节点之间都存在一条边。同一个节点在一条路径序列中 **至多出现一次**。该路径 **至少包含一个** 节点，且不一定经过根节点。

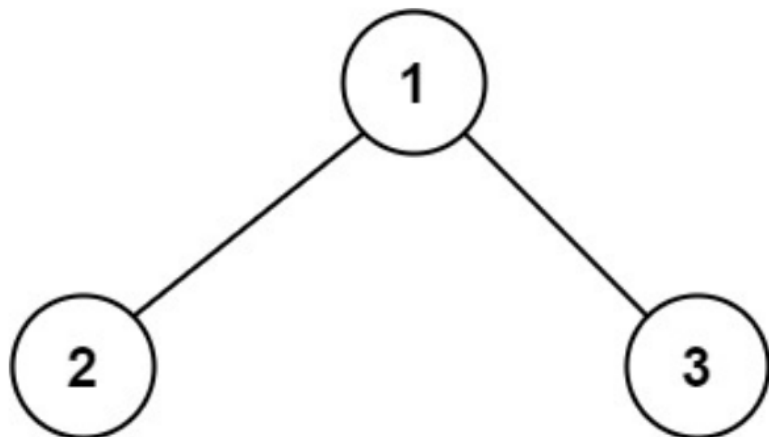
路径和 是路径中各节点值的总和。

给你一个二叉树的根节点 `root`，返回其 **最大路径和**。

提示：

- 树中节点数目范围是 $[1, 3 * 10^4]$
- $-1000 \leq \text{Node.val} \leq 1000$

示例 1:

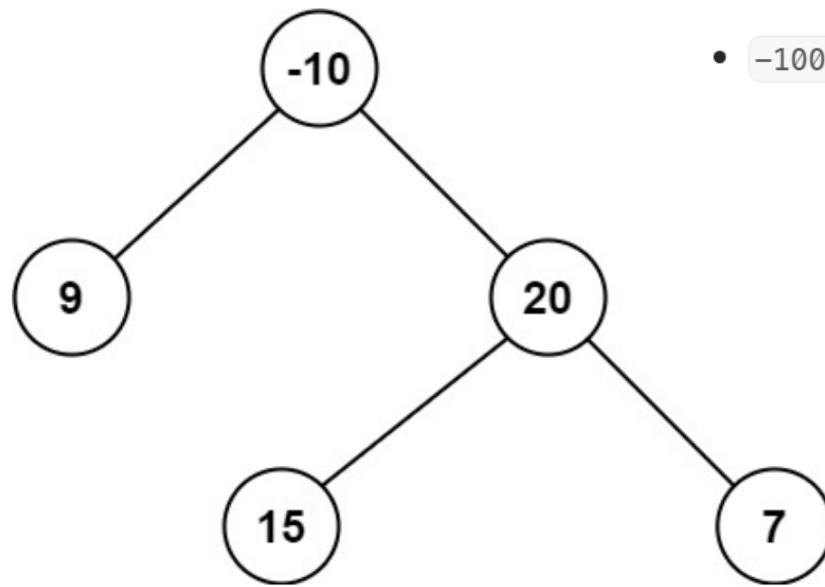


输入: `root = [1,2,3]`

输出: 6

解释: 最优路径是 `2 -> 1 -> 3`，路径和为 `2 + 1 + 3 = 6`

示例 2:



输入: `root = [-10,9,20,null,null,15,7]`

输出: 42

解释: 最优路径是 `15 -> 20 -> 7`，路径和为 `15 + 20 + 7 = 42`

50. 二叉树中的最大路径和 (No.124) HARD

思路

这题和[40. 二叉树的直径]几乎就是同一套代码，只不过这题每个节点带权了而已，因此res更新和函数返回值不太一样。同样用一个函数longestLength返回以root为根节点向下的最大长度（可以一个节点都没有，返回0），在计算过程中同步更新最大路径和。

时间复杂度: $O(n)$

空间复杂度: $O(h)$

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        def longestLength(root):
            # 返回以root为根节点向下的最大路径长度（可为0）
            if not root:
                return 0
            left_len = longestLength(root.left)
            right_len = longestLength(root.right)
            self.res = max(self.res, left_len + right_len + root.val)
            return max(0, max(left_len, right_len) + root.val)

        self.res = -inf
        longestLength(root)
        return self.res
```